

README.md: 1.2 – The Monolithic System Exists

A narrated walkthrough of a TypeScript + Express monolith

README.md: App Overview – What it does

This application is a **small scheduling API**. It lets a client create a scheduling event, submit a user's availability for that event, and list all availability for an event. It also exposes a simple `/health` endpoint so we can confirm the server is alive. This is intentionally tiny so we can see the whole system at once.

README.md: App Overview – What it does *not* do

It does **not** include authentication, persistent storage, or a user interface. It does **not** enforce strict time-zone rules or availability constraints. It does **not** include pagination, caching, or production logging. These omissions are deliberate: the goal is clarity about structure, not completeness.

README.md: What is a monolith?

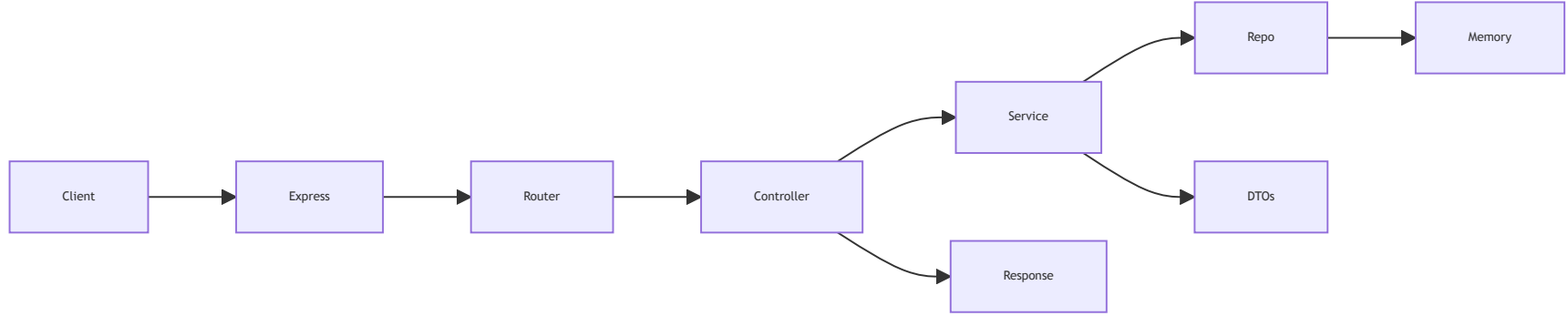
A **monolith** is a single deployable application that contains all core layers—routing, business logic, and data access—in one codebase and usually one process. The benefits are simplicity, shared code, and easy local development. The trade-off is that boundaries are *logical*, not physical: we must design the code structure carefully to keep the layers clean.

README.md: How monoliths are organized

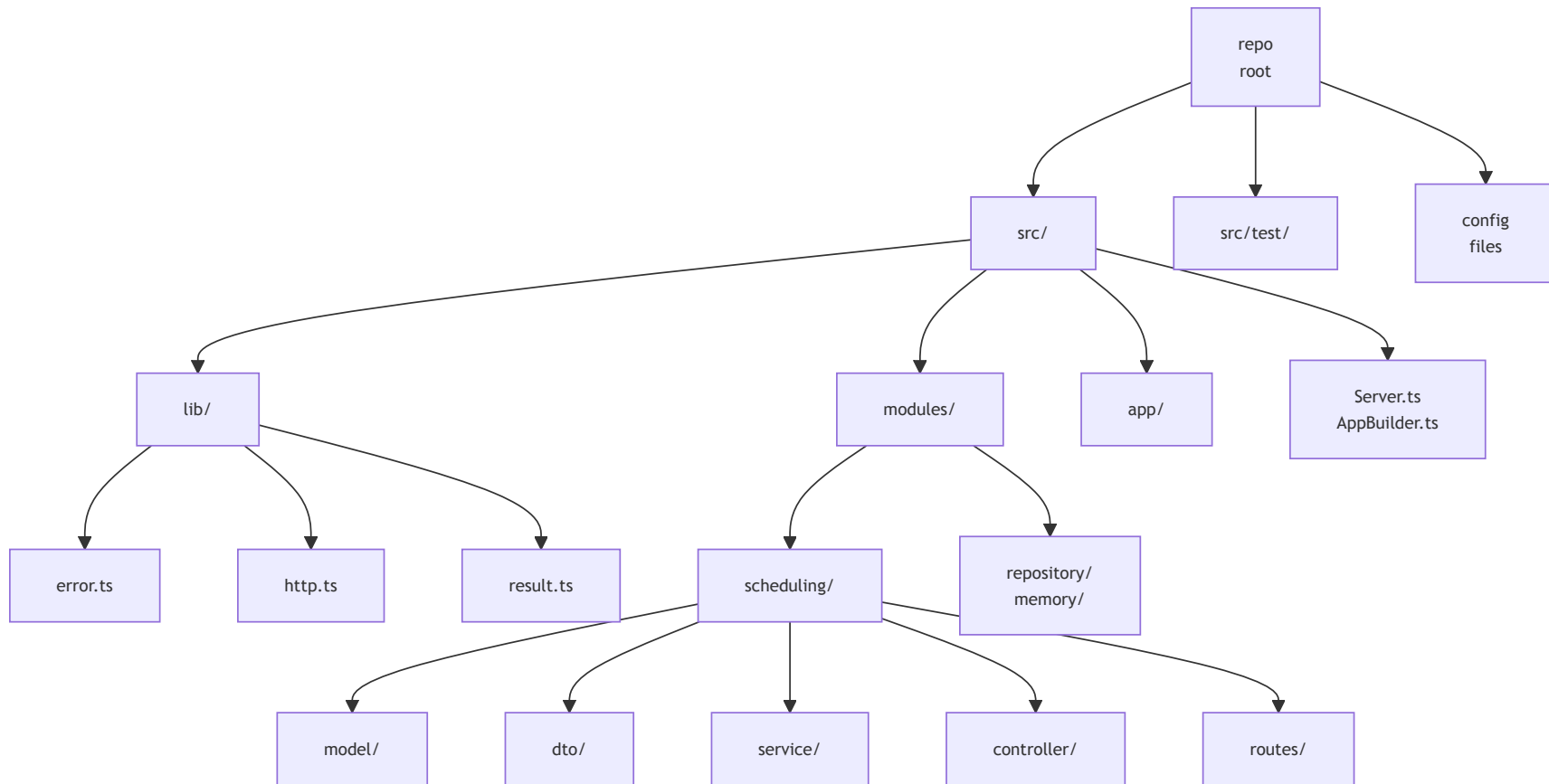
Even in a monolith, we still separate responsibilities:

- **lib/**: shared utilities (errors, results, HTTP helpers).
- **modules/**: domain logic (models, services, repositories).
- **app/**: application wiring (Express middleware and routes).
- **tests/**: behavior verification (integration tests). This lecture follows that same order so the code reads like a story.

README.md: Architecture diagram (high level)



README.md: Code organization diagram



README.md: Request lifecycle at a glance

src/lib/result.ts: Why a Result type?

We want a consistent way to return **success or failure** without throwing exceptions everywhere. A small `Result` type makes the flow explicit: functions either return `ok(value)` or `err(error)`, and callers can respond accordingly. This keeps controllers simple and predictable.

```
// src/lib/result.ts (lines 1-13)
export interface Ok<T> {
  ok: true;
  value: T;
}

export interface Err<E> {
  ok: false;
  error: E;
}

export type Result<T, E> = Ok<T> | Err<E>;
```

src/lib/result.ts: Helper constructors

These helpers avoid repeated object literals. `ok()` and `err()` keep the code readable and consistent across services and controllers.

```
// src/lib/result.ts (lines 15-29)
export function ok<T>(value: T): Ok<T> {
  return { ok: true, value };
}

export function err<E>(error: E): Err<E> {
  return { ok: false, error };
}

export function isOk<T, E>(r: Result<T, E>): r is Ok<T> {
  return r.ok;
}

export function isErr<T, E>(r: Result<T, E>): r is Err<E> {
  return !r.ok;
}
```

src/lib/error.ts: Centralized API errors

A consistent error shape avoids ad-hoc status codes or message strings scattered across the codebase. We define a small set of error codes and provide constructors for each one.

```
// src/lib/error.ts (lines 1-18)
export type ApiErrorCode =
  | "VALIDATION_ERROR"
  | "NOT_FOUND"
  | "CONFLICT"
  | "INTERNAL_ERROR";

export class ApiError extends Error {
  public readonly code: ApiErrorCode;
  public readonly status: number;
  public readonly details?: unknown;

  constructor(args: { code: ApiErrorCode; status: number; message: string; details?: unknown }) {
    super(args.message);
    this.code = args.code;
    this.status = args.status;
    this.details = args.details;
  }
}
```

src/lib/error.ts: Factory methods

These helpers keep error creation consistent and searchable throughout the code.

```
// src/lib/error.ts (lines 19-33)
static validation(message: string, details?: unknown): ApiError {
    return new ApiError({ code: "VALIDATION_ERROR", status: 400, message, details });
}

static notFound(message: string): ApiError {
    return new ApiError({ code: "NOT_FOUND", status: 404, message });
}

static conflict(message: string): ApiError {
    return new ApiError({ code: "CONFLICT", status: 409, message });
}

static internal(message = "Internal error", details?: unknown): ApiError {
    return new ApiError({ code: "INTERNAL_ERROR", status: 500, message, details });
}
}
```

README.md: Async programming & Promise<T> (primer)

Before we see `async` functions in the code, we need a quick mental model. **Asynchronous programming** means a function can start work and *finish later*. In JavaScript/TypeScript, a `Promise<T>` is the object that represents that “future” result: it will eventually **resolve** to a `T`, or **reject** with an error. We use `async / await` to write this in a readable, step-by-step way. We’ll go deeper in a later lecture, but this is enough to follow today’s flow.

src/lib/http.ts: Async safety for Express

Express does not automatically catch rejected promises from async handlers. This wrapper converts any async handler into a safe handler that forwards errors to Express's error middleware.

```
// src/lib/http.ts (lines 1-11)
import type { NextFunction, Request, Response } from "express";
import { ApiError } from "../error";
import type { Result } from "../result";

export function asyncHandler(
  fn: (req: Request, res: Response, next: NextFunction) => Promise<any>,
) {
  return (req: Request, res: Response, next: NextFunction) => {
    Promise.resolve(fn(req, res, next)).catch(next);
  };
}
```

src/lib/http.ts: Send a Result as HTTP

`sendResult` is a tiny adapter from domain logic to HTTP responses. This is the glue between our service layer and our controller layer.

```
// src/lib/http.ts (lines 13-30)
export function sendResult<T>(
  res: Response,
  result: Result<T, ApiError>,
  successStatus = 200,
): void {
  if (result.ok) {
    res.status(successStatus).json(result.value);
    return;
  }

  const e = result.error;
  res.status(e.status).json({
    error: {
      code: e.code,
      message: e.message,
      details: e.details ?? null,
    },
  });
}
```

IEvent.ts: Domain model

Path: `src/modules/scheduling/model/IEvent.ts`

The **domain model** uses `Date` objects because business logic works with real time objects, not strings.

```
// src/modules/scheduling/model/IEvent.ts (lines 1-9)
export default interface IEvent {
  id: string;
  title: string;
  timezone: string;
  startsAt: Date;
  endsAt: Date;
  createdAt: Date;
}
```


IAvailability.ts: Domain model

Path: `src/modules/scheduling/model/IAvailability.ts`

Availability is also represented with `Date` objects for accuracy in logic and comparisons.

```
// src/modules/scheduling/model/IAvailability.ts (lines 1-10)
export default interface IAvailability {
  id: string;
  eventId: string;
  userId: string;
  availableStart: Date;
  availableEnd: Date;
  note?: string;
  createdAt: Date;
}
```

IEventDTO.ts: DTO shape

Path: `src/modules/scheduling/dto/IEventDTO.ts`

A **DTO** (data transfer object) is the shape of data we send over HTTP. Dates become ISO strings.

```
// src/modules/scheduling/dto/IEventDTO.ts (lines 1-11)
export default interface IEventDTO {
  id: string;
  title: string;
  timezone: string;
  startsAt: string; // ISO
  endsAt: string; // ISO
  createdAt: string;
}
```

IAvailabilityDTO.ts: DTO shape

Path: `src/modules/scheduling/dto/IAvailabilityDTO.ts`

Here we also normalize `note` to `null` when absent so the response is consistent.

```
// src/modules/scheduling/dto/IAvailabilityDTO.ts (lines 1-14)
export default interface IAvailabilityDTO {
  id: string;
  eventId: string;
  userId: string;
  availableStart: string; // ISO
  availableEnd: string; // ISO
  note: string | null;
  createdAt: string;
}
```

ICreateEventInput.ts: Input contract

Path: `src/modules/scheduling/service/ICreateEventInput.ts`

Input interfaces describe the shape of **validated** request bodies that flow into the service layer.

```
// src/modules/scheduling/service/ICreateEventInput.ts (lines 1-8)
export default interface ICreateEventInput {
  title: string;
  timezone: string;
  startsAt: string; // ISO
  endsAt: string; // ISO
}
```

ISubmitAvailabilityInput.ts: Input contract

Path: `src/modules/scheduling/service/ISubmitAvailabilityInput.ts`

This contract makes the service signature explicit and stable.

```
// src/modules/scheduling/service/ISubmitAvailabilityInput.ts (lines 1-8)
export default interface ISubmitAvailabilityInput {
  userId: string;
  availableStart: string; // ISO
  availableEnd: string; // ISO
  note?: string;
}
```

ISchedulingRepository.ts: Repository interface

Path: `src/modules/scheduling/repository/ISchedulingRepository.ts`

The service depends on an interface, not a concrete database. This keeps storage swappable.

```
// src/modules/scheduling/repository/ISchedulingRepository.ts (lines 1-30)
```

```
import IEvent from "../model/IEvent";
```

```
import IAvailability from "../model/IAvailability";
```

```
export default interface ISchedulingRepository {
```

```
  createEvent(args: {
```

```
    title: string;
```

```
    timezone: string;
```

```
    startsAt: Date;
```

```
    endsAt: Date;
```

```
  }): Promise<IEvent>;
```

```
  getEventById(eventId: string): Promise<IEvent | null>;
```

```
  upsertAvailability(args: {
```

```
    eventId: string;
```

```
    userId: string;
```

```
    availableStart: Date;
```

```
    availableEnd: Date;
```

```
    note?: string;
```

```
  }): Promise<IAvailability>;
```

```
  listAvailability(eventId: string): Promise<IAvailability[]>;
```

InMemorySchedulingRepo.ts: In-memory store (1/3)

Path: `src/modules/repository/memory/InMemorySchedulingRepo.ts`

We start with an in-memory implementation so we can focus on behavior before databases.

```
// src/modules/repository/memory/InMemorySchedulingRepo.ts (lines 1-16)
import ISchedulingRepository from "../../scheduling/repository/ISchedulingRepository";
import IEvent from "../../scheduling/model/IEvent";
import IAvailability from "../../scheduling/model/IAvailability";
import crypto from "crypto";

export class InMemorySchedulingRepo implements ISchedulingRepository {
  private events: Map<string, IEvent> = new Map();
  private availabilities: Map<string, IAvailability[]> = new Map();
}
```

InMemorySchedulingRepo.ts: In-memory store (2/3)

Path: `src/modules/repository/memory/InMemorySchedulingRepo.ts`

Creating and fetching an event is straightforward: generate IDs, store in a `Map`.

```
// src/modules/repository/memory/InMemorySchedulingRepo.ts (lines 17-37)
async createEvent(args: { title: string; timezone: string; startsAt: Date; endsAt: Date; }): Promise<IEvent> {
  const event: IEvent = {
    id: crypto.randomUUID(),
    title: args.title,
    timezone: args.timezone,
    startsAt: args.startsAt,
    endsAt: args.endsAt,
    createdAt: new Date(),
  };
  this.events.set(event.id, event);
  return event;
}

async getEventById(eventId: string): Promise<IEvent | null> {
  return this.events.get(eventId) || null;
}
```


InMemorySchedulingRepo.ts: In-memory store (3/3)

Path: `src/modules/repository/memory/InMemorySchedulingRepo.ts`

Availability is **upserted** so each user has a single availability per event.

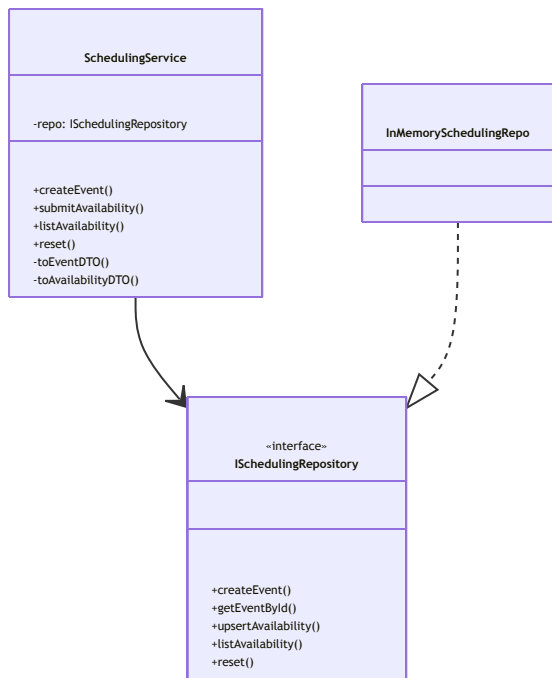
```
// src/modules/repository/memory/InMemorySchedulingRepo.ts (lines 39-79)
async upsertAvailability(args: { eventId: string; userId: string; availableStart: Date; availableEnd: Date; note?: string }) {
  const availabilitiesForEvent = this.availabilities.get(args.eventId) || [];
  let availability = availabilitiesForEvent.find((a) => a.userId === args.userId);

  if (availability) {
    availability.availableStart = args.availableStart;
    availability.availableEnd = args.availableEnd;
    availability.note = args.note;
  } else {
    availability = {
      id: crypto.randomUUID(),
      eventId: args.eventId,
      userId: args.userId,
      availableStart: args.availableStart,
      availableEnd: args.availableEnd,
      note: args.note,
      createdAt: new Date(),
    };
    availabilitiesForEvent.push(availability);
  }
}
```

SchedulingService.ts: Service overview

Path: `src/modules/scheduling/service/SchedulingService.ts`

The service layer is where **business rules** live. It validates time boundaries and converts domain models into DTOs for HTTP responses.



SchedulingService.ts: Imports & ctor

Path: `src/modules/scheduling/service/SchedulingService.ts`

```
// src/modules/scheduling/service/SchedulingService.ts (lines 1-14)
import type ISchedulingRepository from "../repository/ISchedulingRepository";
import { ApiError } from "../../lib/error";
import { Result, err, ok } from "../../lib/result";
import ICreateEventInput from "../ICreateEventInput";
import ISubmitAvailabilityInput from "../ISubmitAvailabilityInput";
import IEventDTO from "../dto/IEventDTO";
import IAvailabilityDTO from "../dto/IAvailabilityDTO";

export default class SchedulingService {
  private readonly repo: ISchedulingRepository;

  constructor(repo: ISchedulingRepository) {
    this.repo = repo;
  }
}
```

SchedulingService.ts: Create event validation

Path: `src/modules/scheduling/service/SchedulingService.ts`

We parse ISO strings into `Date` and enforce a basic ordering rule.

```
// src/modules/scheduling/service/SchedulingService.ts (lines 16-26)
async createEvent(input: ICreateEventInput): Promise<Result<IEventDTO, ApiError>> {
  const startsAt = new Date(input.startsAt);
  const endsAt = new Date(input.endsAt);

  if (Number.isNaN(startsAt.getTime()) || Number.isNaN(endsAt.getTime())) {
    return err(ApiError.validation("startsAt and endsAt must be valid ISO datetimes"));
  }
  if (endsAt ≤ startsAt) {
    return err(ApiError.validation("endsAt must be after startsAt"));
  }
}
```

SchedulingService.ts: Create event persistence

Path: `src/modules/scheduling/service/SchedulingService.ts`

Now we call the repository and map the response to a DTO.

```
// src/modules/scheduling/service/SchedulingService.ts (lines 28-35)
    const event = await this.repo.createEvent({
      title: input.title,
      timezone: input.timezone,
      startsAt,
      endsAt
    });

    return ok(this.toEventDTO(event));
  }
```

SchedulingService.ts: Submit availability validation

Path: `src/modules/scheduling/service/SchedulingService.ts`

We check that the event exists and ensure the time interval makes sense.

```
// src/modules/scheduling/service/SchedulingService.ts (lines 38-53)
async submitAvailability(eventId: string, input: ISubmitAvailabilityInput): Promise<Result<IAvailabilityDTO, ApiError>> {
  const event = await this.repo.getEventById(eventId);
  if (!event) return err(ApiError.notFound(`No event exists with id ${eventId}`));

  const start = new Date(input.availableStart);
  const end = new Date(input.availableEnd);

  if (Number.isNaN(start.getTime()) || Number.isNaN(end.getTime())) {
    return err(ApiError.validation("availableStart and availableEnd must be valid ISO datetimes"));
  }
  if (end ≤ start) {
    return err(ApiError.validation("availableEnd must be after availableStart"));
  }
}
```

SchedulingService.ts: Submit availability save

Path: `src/modules/scheduling/service/SchedulingService.ts`

The **upsert** ensures each user has one availability per event.

```
// src/modules/scheduling/service/SchedulingService.ts (lines 55-67)
    const saved = await this.repo.upsertAvailability({
      eventId,
      userId: input.userId,
      availableStart: start,
      availableEnd: end,
      note: input.note,
    });

    return ok(this.toAvailabilityDTO(saved));
  }
```

SchedulingService.ts: List availability & reset

Path: `src/modules/scheduling/service/SchedulingService.ts`

Listing joins event info with availability records. Reset clears data for tests.

```
// src/modules/scheduling/service/SchedulingService.ts (lines 69-83)
  async listAvailability(eventId: string): Promise<Result<{ event: IEventDTO; availability: IAvailabilityDTO[] }, ApiError>> {
    const event = await this.repo.getEventById(eventId);
    if (!event) return err(ApiError.notFound(`No event exists with id ${eventId}`));

    const rows = await this.repo.listAvailability(eventId);

    return ok({ event: this.toEventDTO(event), availability: rows.map((r) => this.toAvailabilityDTO(r)) });
  }

  async reset(): Promise<Result<void, ApiError>> {
    await this.repo.reset();
    return ok(void 0);
  }
```


SchedulingService.ts: DTO mapping

Path: `src/modules/scheduling/service/SchedulingService.ts`

Mapping keeps the HTTP layer free from `Date` objects.

```
// src/modules/scheduling/service/SchedulingService.ts (lines 86-115 excerpt)
private toEventDTO(event: { id: string; title: string; timezone: string; startsAt: Date; endsAt: Date; createdAt: Date }) {
  return {
    id: event.id,
    title: event.title,
    timezone: event.timezone,
    startsAt: event.startsAt.toISOString(),
    endsAt: event.endsAt.toISOString(),
    createdAt: event.createdAt.toISOString()
  };
}

private toAvailabilityDTO(row: { id: string; eventId: string; userId: string; availableStart: Date; availableEnd: Date; note: string }) {
  return {
    id: row.id,
    eventId: row.eventId,
    userId: row.userId,
    availableStart: row.availableStart.toISOString(),
    availableEnd: row.availableEnd.toISOString(),
    note: row.note ?? null,
  };
}
```

SchedulingService.ts: Sequence – Create Event

Path: `src/modules/scheduling/service/SchedulingService.ts`

SchedulingService.ts: Sequence – Submit Availability

Path: `src/modules/scheduling/service/SchedulingService.ts`

SchedulingControllerValidator.ts: Zod validation

Path: `src/modules/scheduling/controller/SchedulingControllerValidator.ts`

Zod validates request shapes at the boundary. It doesn't enforce all business rules; it ensures types and basic constraints before the service runs.

```
// src/modules/scheduling/controller/SchedulingControllerValidator.ts (lines 1-21)
import { z } from "zod";

export default class SchedulingControllerValidator {
  static createEventSchema = z.object({
    title: z.string().min(1).max(200),
    timezone: z.string().min(1).max(100),
    startsAt: z.iso.datetime(),
    endsAt: z.iso.datetime(),
  });

  static submitAvailabilitySchema = z.object({
    userId: z.string().min(1).max(200),
    availableStart: z.iso.datetime(),
    availableEnd: z.iso.datetime(),
    note: z.string().max(500).optional(),
  });
}
```

SchedulingController.ts: Controller role

Path: `src/modules/scheduling/controller/SchedulingController.ts`

Controllers connect HTTP to services. They validate inputs, call the service, and translate the Result into an HTTP response.

```
// src/modules/scheduling/controller/SchedulingController.ts (lines 1-13)
import type { Request, Response } from "express";
import SchedulingService from "../service/SchedulingService";
import { ApiError } from "../../lib/error";
import { sendResult } from "../../lib/http";
import { err, ok } from "../../lib/result";
import V from "../SchedulingControllerValidator";

export default class SchedulingController {
  private readonly service: SchedulingService;

  constructor(service: SchedulingService) {
    this.service = service;
  }
}
```

SchedulingController.ts: Create event handler

Path: `src/modules/scheduling/controller/SchedulingController.ts`

Notice how little business logic lives here; it's just validation plus wiring.

```
// src/modules/scheduling/controller/SchedulingController.ts (lines 15-26)
async createEvent(req: Request, res: Response): Promise<void> {
  const parsed = V.createEventSchema.safeParse(req.body);
  if (!parsed.success) {
    sendResult(res, err(ApiError.validation("Invalid request body", parsed.error.flatten())), 400);
    return;
  }

  const result = await this.service.createEvent(parsed.data);
  sendResult(res, result, 201);
}
```

SchedulingController.ts: Submit availability (1/2)

Path: `src/modules/scheduling/controller/SchedulingController.ts`

Validate the route parameter before using it.

```
// src/modules/scheduling/controller/SchedulingController.ts (lines 28-37)
async submitAvailability(req: Request, res: Response): Promise<void> {
  if (!V.eventIdSchema.safeParse(req.params.eventId).success) {
    sendResult(res, err(ApiError.validation("Missing eventId")), 400);
    return;
  }

  const eventId = req.params.eventId as string;
```

SchedulingController.ts: Submit availability (2/2)

Path: `src/modules/scheduling/controller/SchedulingController.ts`

Validate the body and call the service.

```
// src/modules/scheduling/controller/SchedulingController.ts (lines 38-46)
const parsed = V.submitAvailabilitySchema.safeParse(req.body);
if (!parsed.success) {
  sendResult(res, err(ApiError.validation("Invalid request body", parsed.error.flatten())), 400);
  return;
}

const result = await this.service.submitAvailability(eventId, parsed.data);
sendResult(res, result, 200);
}
```


SchedulingController.ts: List availability

Path: `src/modules/scheduling/controller/SchedulingController.ts`

A standard read path: validate, call service, return result.

```
// src/modules/scheduling/controller/SchedulingController.ts (lines 48-59)
async listAvailability(req: Request, res: Response): Promise<void> {
  if (!V.eventIdSchema.safeParse(req.params.eventId).success) {
    sendResult(res, err(ApiError.validation("Missing eventId")), 400);
    return;
  }

  const eventId = req.params.eventId as string;

  const result = await this.service.listAvailability(eventId);
  sendResult(res, result, 200);
}
```

SchedulingController.ts: Reset endpoint

Path: `src/modules/scheduling/controller/SchedulingController.ts`

This endpoint exists for tests only. It clears state between test cases.

```
// src/modules/scheduling/controller/SchedulingController.ts (lines 61-64)
  async reset(req: Request, res: Response): Promise<void> {
    await this.service.reset();
    sendResult(res, ok({ message: "Reset successful" }), 200);
  }
}
```

SchedulingRoutesBuilder.ts: Bind handlers

Path: `src/modules/scheduling/routes/SchedulingRoutesBuilder.ts`

Binding ensures the controller instance remains `this` inside each method.

```
// src/modules/scheduling/routes/SchedulingRoutesBuilder.ts (lines 1-15)
import { Router } from "express";
import SchedulingController from "../controller/SchedulingController";
import { asyncHandler } from "../../lib/http";

export default class SchedulingRoutesBuilder {
  static build(controller: SchedulingController): Router {
    const router = Router();

    const createEventHandler = controller.createEvent.bind(controller);
    const submitAvailabilityHandler = controller.submitAvailability.bind(controller);
    const listAvailabilityHandler = controller.listAvailability.bind(controller);
    const resetHandler = controller.reset.bind(controller);
```

SchedulingRoutesBuilder.ts: Register routes

Path: `src/modules/scheduling/routes/SchedulingRoutesBuilder.ts`

All routes are mounted under `/api` by the app router.

```
// src/modules/scheduling/routes/SchedulingRoutesBuilder.ts (lines 17-23)
    router.post("/events", asyncHandler(createEventHandler));
    router.post("/events/:eventId/availability", asyncHandler(submitAvailabilityHandler));
    router.get("/events/:eventId/availability", asyncHandler(listAvailabilityHandler));
    router.post("/reset", asyncHandler(resetHandler));

    return router;
  }
}
```

src/app/router.ts: Imports

The router is the **application boundary**: it combines middleware and routes.

```
// src/app/router.ts (lines 1-4)
import express, {
  type Express,
  type NextFunction,
  type Request,
  type Response,
} from "express";
import { ApiError } from "../lib/error";
import SchedulingRoutesBuilder from "../modules/scheduling/routes/SchedulingRoutesBuilder";
import SchedulingController from "../modules/scheduling/controller/SchedulingController";
```

src/app/router.ts: Initialize Express

The constructor builds the Express app and installs middleware.

```
// src/app/router.ts (lines 24-30)
constructor(args: { schedulingController: SchedulingController }) {
  // 1. Initialize the Express app
  this.expressApp = express();

  // 2. Common middleware
  this.expressApp.use(express.json());
```

src/app/router.ts: Health & route mount

A health check is simple, but useful in production too.

```
// src/app/router.ts (lines 31-41)
// Health check
this.expressApp.get("/health", (_req, res) => res.json({ ok: true }));

// Scheduling API routes
this.expressApp.use(
  "/api",
  SchedulingRoutesBuilder.build(args.schedulingController),
);
```

src/app/router.ts: 404 handler

A consistent 404 response keeps client behavior predictable.

```
// src/app/router.ts (lines 44-48)
// Not found handler
this.expressApp.use((_req, res) => {
  res
    .status(404)
    .json({ error: { code: "NOT_FOUND", message: "Route not found" } });
});
```


src/app/router.ts: Error handler

Express error middleware has four parameters. We use it to shape all server errors.

```
// src/app/router.ts (lines 50-73)
this.expressApp.use((error: unknown, _req: Request, res: Response, _next: NextFunction) => {
  if (error instanceof ApiError) {
    res.status(error.status).json({
      error: {
        code: error.code,
        message: error.message,
        details: error.details ?? null
      }
    });
    return;
  }

  res.status(500).json({
    error: {
      code: "INTERNAL_ERROR",
      message: "Unexpected error",
      details: String(error)
    }
  });
});
}
```

src/app/router.ts: Expose Express

This method lets the server and tests use the configured app instance.

```
// src/app/router.ts (lines 77-80)
getExpress(): Express {
  return this.expressApp;
}
```

src/AppBuilder.ts: Wiring the app

AppBuilder is the *composition root* of the monolith. It makes the object graph explicit.

```
// src/AppBuilder.ts (lines 1-4)
import { AppRouter } from "../app/router";
import { InMemorySchedulingRepo } from "../modules/repository/memory/InMemorySchedulingRepo";
import SchedulingService from "../modules/scheduling/service/SchedulingService";
import SchedulingController from "../modules/scheduling/controller/SchedulingController";
```

src/AppBuilder.ts: Constructing the graph

Here is the single place we can swap storage or service implementations.

```
// src/AppBuilder.ts (lines 14-21)
export default class AppBuilder {
  static build(): AppRouter {
    const repo = new InMemorySchedulingRepo();
    const service = new SchedulingService(repo);
    const controller = new SchedulingController(service);

    return new AppRouter({ schedulingController: controller });
  }
}
```

src/Server.ts: Bootstrapping the monolith

The server file is intentionally tiny: its entire job is to start listening.

```
// src/Server.ts (lines 1-15)
import AppBuilder from "../AppBuilder";

class Server {
  static start() {
    const port = Number(process.env.PORT ?? 3000);

    const app = AppBuilder.build().getExpress();

    console.log(`Database URL: ${process.env.DATABASE_URL}`);

    app.listen(port, () => {
      console.log(`Listening on http://localhost:${port}`);
    });
  }
}
```

src/Server.ts: Main entry point

```
// src/Server.ts (lines 18-19)  
Server.start();
```

scheduling.int.test.ts: Test setup

Path: `src/test/scheduling.int.test.ts`

Integration tests run the real Express app in memory with Supertest.

```
// src/test/scheduling.int.test.ts (lines 1-13)
import request from "supertest";
import AppBuilder from "../AppBuilder";

describe("Scheduling API (integration)", () => {
  const app = AppBuilder.build().getExpress();

  beforeEach(async () => {
    await request(app).post("/api/reset").expect(200);
  });
```

scheduling.int.test.ts: Happy path

Path: `src/test/scheduling.int.test.ts`

The test shows the complete workflow: create → submit availability → list.

```
// src/test/scheduling.int.test.ts (lines 14-52 excerpt)
test("create event, submit availability, list availability, data survives across requests", async () => {
  const createRes = await request(app)
    .post("/api/events")
    .send({
      title: "Office Hours Planning",
      timezone: "America/New_York",
      startsAt: "2026-02-01T14:00:00.000Z",
      endsAt: "2026-02-01T16:00:00.000Z",
    })
    .expect(201);

  const eventId = createRes.body.id as string;

  const availRes = await request(app)
    .post(`/api/events/${eventId}/availability`)
    .send({
      userId: "alice",
      availableStart: "2026-02-01T14:30:00.000Z",
      availableEnd: "2026-02-01T15:15:00.000Z",
    })
```


scheduling.int.test.ts: Error cases

Path: `src/test/scheduling.int.test.ts`

Validation and not-found behavior are part of the contract.

```
// src/test/scheduling.int.test.ts (lines 54-71)
test("unknown event returns 404", async () => {
  const res = await request(AppBuilder.build().getExpress()).get("/api/events/does-not-exist/availability").expect(
    expect(res.body.error.code).toBe("NOT_FOUND");
  });

test("validation rejects bad time interval", async () => {
  const createRes = await request(AppBuilder.build().getExpress())
    .post("/api/events")
    .send({
      title: "Bad Event",
      timezone: "America/New_York",
      startsAt: "2026-02-01T16:00:00.000Z",
      endsAt: "2026-02-01T14:00:00.000Z"
    })
    .expect(400);

  expect(createRes.body.error.code).toBe("VALIDATION_ERROR");
});
```

test.http: Manual HTTP checks

Path: `src/test/test.http`

This file is a friendly way to experiment without a full client app.

```
// src/test/test.http (lines 1-15)
## Create Event
POST http://localhost:3000/api/events
Content-Type: application/json

{
  "title": "Test Event",
  "timezone": "America/New_York",
  "startsAt": "2026-01-29T00:00:00Z",
  "endsAt": "2026-01-29T01:00:00Z"
}
```

package.json: Scripts

These scripts are the control panel for development, testing, and production builds.

```
// package.json (scripts excerpt)
"scripts": {
  "dev1": "dotenv -- ts-node-dev --respawn --transpile-only src/Server.ts",
  "dev": "dotenv -- node --watch -r ts-node/register src/server.ts",
  "build": "tsc -p tsconfig.json",
  "start": "node --env-file=.env dist/Server.js",
  "test": "dotenv -- jest --runInBand"
}
```

package.json: Dependencies

These are the building blocks: Express for HTTP, Zod for validation, Jest/Supertest for tests.

```
// package.json (deps excerpt)
"dependencies": {
  "dotenv-cli": "^11.0.0",
  "express": "^5.2.1",
  "zod": "^4.3.6"
},
"devDependencies": {
  "@types/express": "^5.0.6",
  "@types/jest": "^30.0.0",
  "@types/node": "^25.1.0",
  "@types/supertest": "^6.0.3",
  "jest": "^30.2.0",
  "supertest": "^7.2.2",
  "ts-jest": "^29.4.6",
  "ts-node-dev": "^2.0.0",
  "typescript": "^5.9.3"
}
```

tsconfig.json: TypeScript compiler

tsconfig.json defines how TypeScript compiles to JavaScript and how strict type-checking is.

```
// tsconfig.json
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["ES2022"],
    "module": "commonjs",
    "moduleResolution": "node",
    "rootDir": "src",
    "outDir": "dist",
    "strict": true,
    "esModuleInterop": true,
    "forceConsistentCasingInFileNames": true,
    "skipLibCheck": true
  }
}
```

jest.config.ts: Test runner configuration

Jest uses `ts-jest` to execute TypeScript test files directly.

```
// jest.config.ts
import type { Config } from "jest";

const config: Config = {
  preset: "ts-jest",
  testEnvironment: "node",
  testMatch: ["**/*.test.ts"],
  clearMocks: true,
  verbose: true,
};

export default config;
```

README.md: References – Languages & Runtime

- TypeScript Handbook
- JavaScript on MDN
- Node.js Documentation
- npm Documentation

README.md: References – TypeScript concepts

- Interfaces
- Union Types
- Generics
- Modules

README.md: References – JavaScript concepts

- Promises
- async/await
- Date
- Map
- Classes

README.md: References – HTTP & APIs

- HTTP overview (MDN)
- HTTP methods (MDN)
- JSON (MDN)
- ISO 8601 date format

README.md: References – Express

- Express Documentation
- Express Routing
- Express Error Handling
- Express Middleware

README.md: References – Validation & Testing

- Zod Documentation
- Zod `safeParse`
- Jest Documentation
- Supertest GitHub

README.md: References – Tooling

- [dotenv-cli GitHub](#)
- [ts-jest Documentation](#)
- [ts-node-dev GitHub](#)
- [Slidev Documentation](#)

README.md: Closing note

This monolith is intentionally small, but it is **structurally complete**: a clean boundary between HTTP, validation, business rules, and storage. The next step is not to abandon the monolith—it is to **evolve** it, one well-designed boundary at a time.